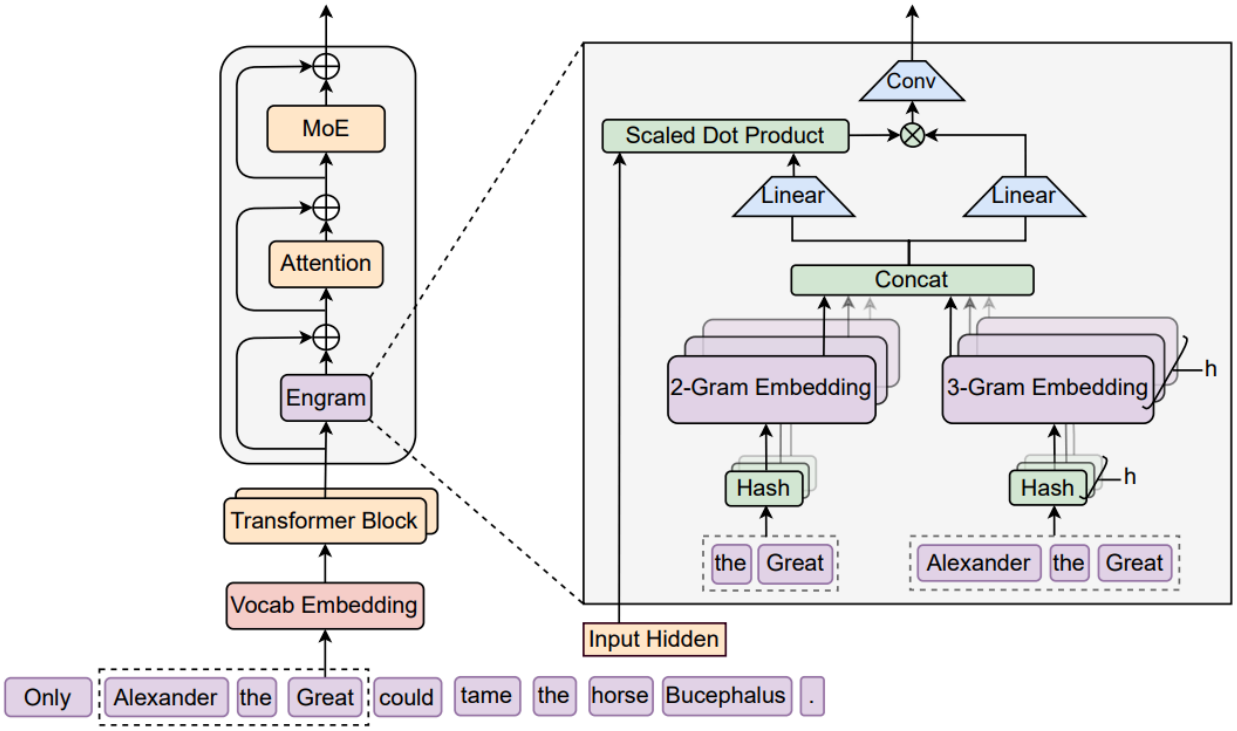
Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models (Engram from Deepseek v4)

- The Mixture-of-Experts (MoE) design has improved the capacity of the model. However, the model still lacks the native primitive of looking for knowledge quickly. That is to say, *the model cannot gain the semantics of knowledge until it reaches the very end of inference: a lot of layers and computes are wasted.*
- An *Engram* module is to solve this problem. By looking $n$tokens back, harsh, and looking up the table, the model will gain the ability to know the semantic information of the tokens during the early stage of inference, and hence the rest layers are for deeper understanding of the context.



- Engram: Given an input sequence $X = (x_1, \ldots, x_T)$and the hidden state of layer l: $\mathbf{H}^{(l)} \in \mathbb{R}^{T \times d}$, the module processes each position t in two functional phases: **retrieval** and **fusion**.
  1. **Tokenizer compression**: convert each token to its equivalence using NFKC, for example, `apple = Apple = APPLE`, and apply a surjective function $P$to each token to assign each token a canonical ID: $x'_t = P(x_t)$. In the end, we have suffix N-gram $g_{t,n} = (x'_{t-n+1}, \ldots, x'_t)$.
  2. **Multi-Head Hashing**: it is impossible to assign all possible N-gram combinations an identical ID and save it on the table (as the VRAM is limited). We fixed each table $\mathbf{E}$to a reasonable *prime size* M. Inevitably, collision will happen. To solve this, for each $n$gram, apply Kharsh heads with their own pre-determined and unique harsh function $\phi_{n,k}(\cdot)$ that will return an integer as the index, and each head has its own table $\mathbf{E}_{n,k}$with its own size $M_{n,k}$, where

     $$\mathbf{e}_{t,n,k} = \mathbf{E}_{n,k}[\phi_{n,k}(g_{t,n})] \text{ for } k \in \{1, \ldots, K\} \text{ and } n \in \{2, \ldots, N\}$$

     - *This is to say, if collision happens in Head #1 (for example, both `New York` and `Hot Dog` are mapped to the same ID in table $\mathbf{E}_{2,1}$), Head #2 and its table may mitigate this.*

- *The vectors in $\mathbf{E}$are embedded semantic information of the current-query N-grams.*

Then, we construct the final memory vector by concactination:

$$\mathbf{e}_t \triangleq \Big\|_{n=2}^{N} \Big\|_{k=1}^{K} \mathbf{e}_{t,n,k}$$

3. **Context-aware Gating**: we shall *selectively* use the queried semantic information. To achieve this, the current hidden state, $\mathbf{h}_t$ is used to calculate a score as the weight (namely, similarity) with $\mathbf{e}_t$, similar to the mechanism of attention. Before that, project $\mathbf{e}_t$with two trainable matrices:

   $$\mathbf{k}_t = \mathbf{W}_K \mathbf{e}_t, \quad \mathbf{v}_t = \mathbf{W}_V \mathbf{e}_t$$

   Calculate the gating score with RMSNorm to ensure stability.

   $$\alpha_t = \sigma\left(\frac{\mathrm{RMSNorm}(\mathbf{h}_t)^\top \mathrm{RMSNorm}(\mathbf{k}_t)}{\sqrt{d}}\right), \alpha_t \in (0,1)$$

   The gated output is $\tilde{v} = \alpha_t \mathbf{v}_t$, where an $\alpha_t \approx 1$means the N-gram matches the queried semantic vector. Let $\tilde{\mathbf{V}} \in \mathbb{R}^{T \times d}$ denote the sequence of gated values; to expand the receptive field and enhance the model's non-linearity, we apply a 1-D CNN to $\tilde{\mathbf{V}}$row by row:

   $$\mathbf{Y} = \mathrm{SiLU}\left(\mathrm{Conv1D}(\mathrm{RMSNorm}(\tilde{\mathbf{V}}))\right) + \tilde{\mathbf{V}}$$

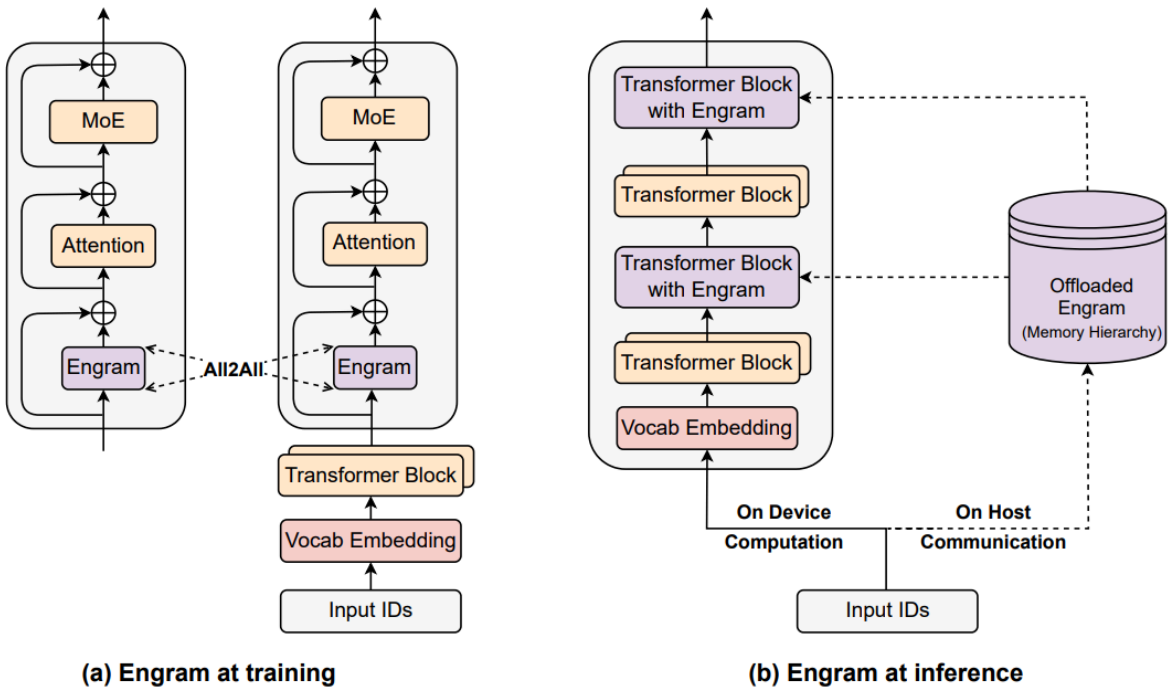
   Add to the backbone via a residual connection:

   $$\mathbf{H}^{(\ell)} \leftarrow \mathbf{H}^{(\ell)} + \mathbf{Y}$$

4. For multi-branch architecture with Mbranches, they share the same sets of tables $\mathbf{E}_{n,k}$. As the information in the tables is objective, which means $\mathbf{e}_t$is always the ground truth (e.g., D.C. is the capital of the U.S.), and $\mathbf{W}_V$is only for projection, $\mathbf{W}_V$is also shared to improve efficiency. However, before calculating the gating score, each branch will have its own $\mathbf{W}_K^m$as each branch $m$may take care of a different perspective (e.g., Branch #1 only cares about "location", and with $\mathbf{W}_K^1$and its own hidden state, a larger value will be produced for "New York"; Branch #2 only cares "mood", then it will produce a value close to 0 for "New York"). In the end, the gated score for each branch will become:

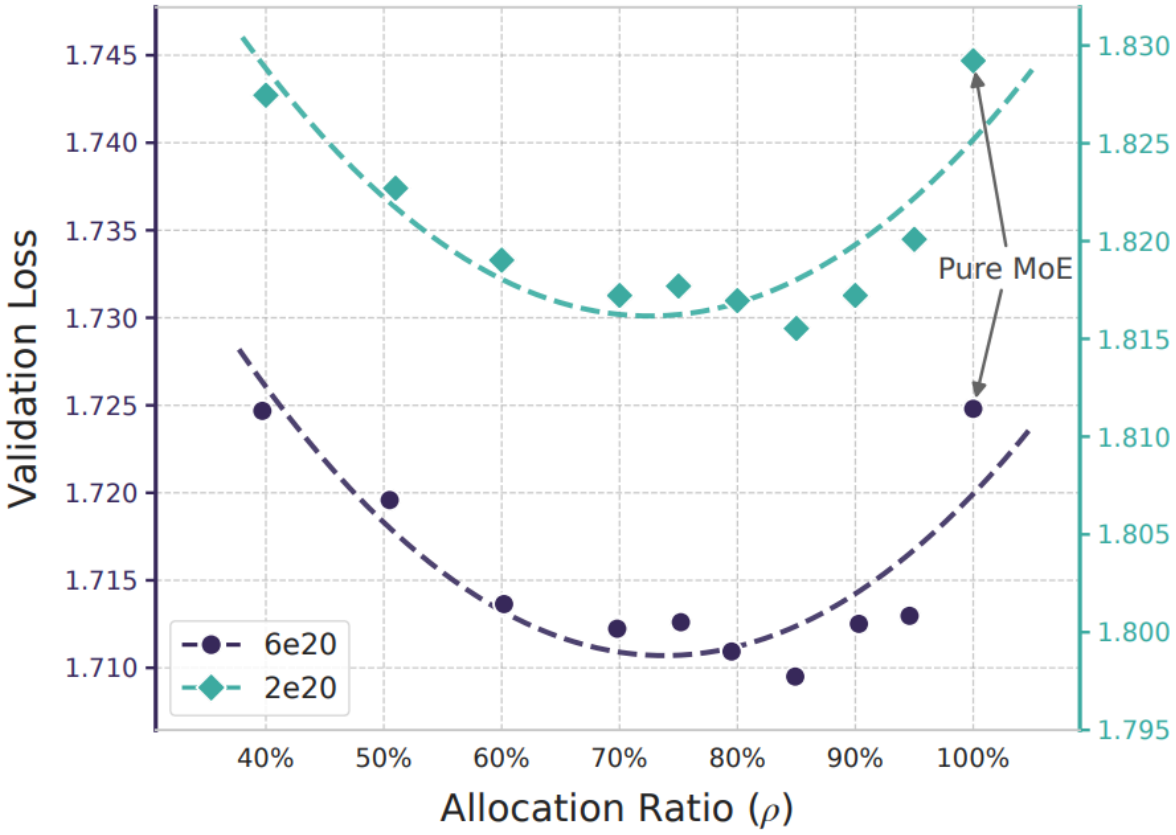
   $$\alpha_t^{(m)} = \sigma\left(\frac{\mathrm{RMSNorm}(\mathbf{h}_t^{(m)})^\top \mathrm{RMSNorm}(\mathbf{W}_K^{(m)} \mathbf{e}_t)}{\sqrt{d}}\right)$$

- Training and Inference



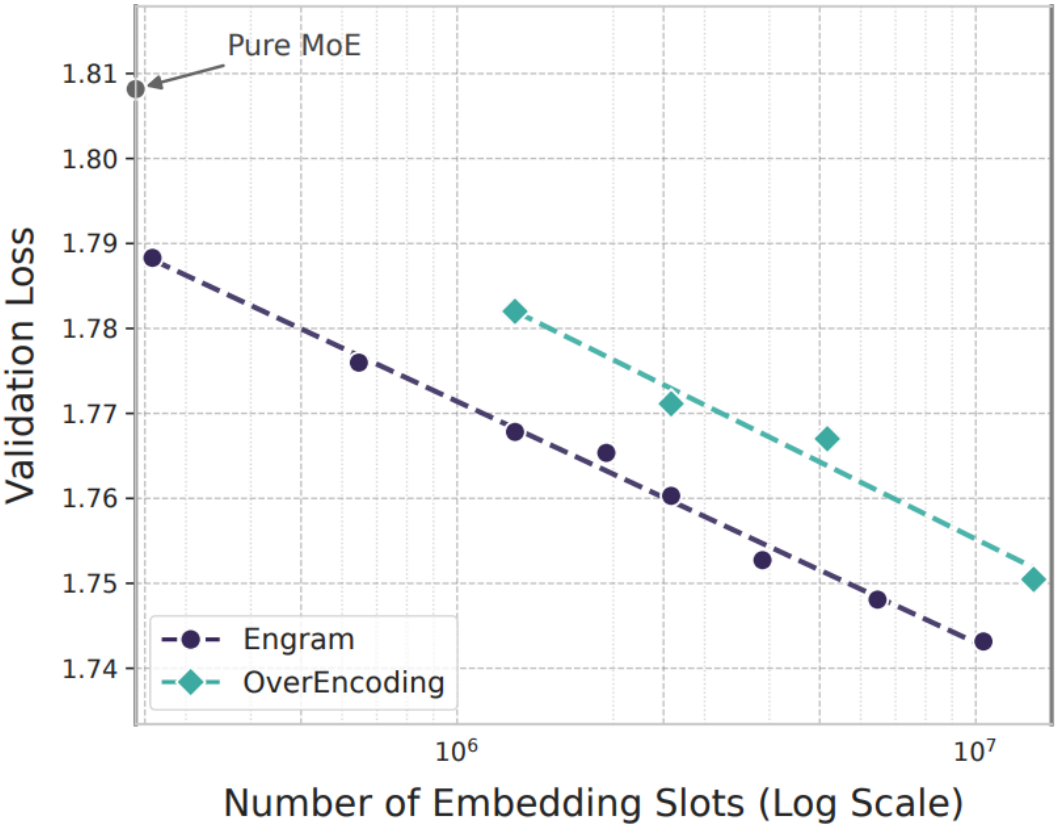
- During the train time, the table is shared among the GPUs in shards, and the parameter will be updated.
- During the inference time, Engram is pre-fetched when the GPU is calculating previous layers to mask the latency. Plus, because of the deterministic property of the table indices, the fetch of the relevant part of the table can happen asynchronizely (`'New York' <-> 0x10034d3`, we simply fetch the table parts by knowing the token).
- Where should the engram module be placed?
  - Shallow layers: the model can know the semantic information earlier and save the deeper layers for detailed understanding of the context.
  - Deep layers: the time for the system to fetch the table from RAM to VRAM is longer, so the latency is better masked.
  - To balance the tradeoff, the engram module is normally placed at the 2nd layer.

- Scaling Laws and Sparsity Allocation
  - Let a model have unallocated parameters P_{sparse}, which can be used either as MoE or engram. How do we allocate the parameters to MoE and to engram to ensure the best performance?



- It shows a U-shaped pattern. When **~80%** of the parameter budget is allocated to the MoE, the model performs the best.

- How does the size of the engram table affect the performance of the model, given that the # of MoE parameters is constant?



- It shows a **decreasing Linear in Log-space** pattern in *validation loss vs size of the engram table*.

- Experiments
    1. Large Scale Pre-training
        - This section validates architecture's performance on multi-billion parameter models under strict resource constraints.
        - Experimental Setup: The authors compared three models: Dense-4B (baseline), MoE-27B, and Engram-27B. Crucially, Engram-27B was designed to be strictly iso-parameter and iso-FLOPs with the MoE-27B baseline by reallocating parameters from routed experts to the Engram memory.
        - Performance Gains: Engram-27B achieved superior performance over the MoE baseline across diverse domains.
        - Surprising Findings: While the memory module was expected to boost knowledge tasks (e.g., MMLU +3.4; CMMLU +4.0), the largest gains were observed in general reasoning (e.g., BBH +5.0) and code/math domains (e.g., HumanEval +3.0; MATH +2.4).
        - Scaling: A larger model, Engram-40B, showed further reductions in loss, indicating that the expanded memory capacity had not yet fully saturated.
    2. Long Context Training
        - This section tests the hypothesis that offloading local dependencies to lookups frees up attention capacity for global context.
        - Setup: Models underwent context extension training (up to 32k tokens) using YaRN. Comparisons were made under "Iso-Loss" (matching pre-training loss) and "Iso-FLOPs" settings to isolate architectural benefits.
        - Retrieval Superiority: Engram-27B substantially outperformed the baseline on long-context retrieval tasks. For example, on Multi-Query NIAH (Needle In A Haystack), Engram scored 97.0 compared to MoE's 84.2.
        - Efficiency: Even an early-stopped Engram checkpoint (trained for fewer steps) outperformed the fully trained MoE baseline on the RULER benchmark, demonstrating intrinsic architectural superiority for long contexts.
    3. Analysis & Mechanisms (Section 6)
        - This section investigates why Engram works and its system-level feasibility.
        - 6.1 Effective Depth (Mechanism)
            - Accelerated Convergence: Using LogitLens, the authors found that Engram models reach valid prediction distributions in earlier layers compared to MoE, suggesting faster feature composition.
            - Deepening the Network: CKA (Centered Kernel Alignment) analysis revealed that shallow layers in Engram are functionally similar to much deeper layers in the MoE baseline (e.g., Engram Layer 5 aligns with MoE Layer 12). This confirms that Engram relieves early layers from static reconstruction, effectively increasing the model's depth for complex reasoning.
        - 6.2 Structural Ablations
            - Optimal Placement: Inserting Engram in early layers (e.g., Layer 2) is optimal. This allows the model to offload local pattern reconstruction early, before the backbone expends computational depth.

        - Key Components: The most critical design choices are branch-specific fusion, context-aware gating, and tokenizer compression. Removing any of these leads to significant performance regression.
    - 6.3 Sensitivity Analysis (What is stored?)
        - Functional Dichotomy: When the Engram module is suppressed during inference:
        - Factual Knowledge collapses (e.g., TriviaQA performance drops to 29%), confirming Engram is the primary store for parametric knowledge.
        - Reading Comprehension remains resilient (e.g., C3 retains 93%), indicating these tasks rely primarily on the backbone's attention mechanism.
    - 6.4 System Efficiency
        - Prefetching: Because Engram's addressing is deterministic (dependent only on input tokens), memory access can be computed ahead of time.
        - Hardware-Awareness: Offloading a massive 100B-parameter embedding table to host memory (CPU RAM) incurred negligible inference overhead (<3%) due to asynchronous prefetching. This proves architecture can scale memory capacity far beyond GPU limits.
- Work Cited
    - Paper
    - Chat
    - Codes